PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula 5 - Introdução à Modelagem Arquitetural

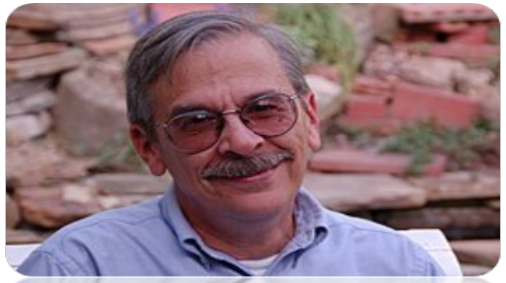
Prof.: Filipe Nhimi

Agenda

- ❑ Conceitos Base
- ❑ Correção do Exercício Prático I – Modelagem Arquitetural (PUC Connect)
- ❑ Conclusão

O que é Modelagem Arquitetural?

"Segundo Bass, Clements e Kazman, modelagem arquitetural é o processo de **representar** a **estrutura do sistema** por meio de **elementos, relações e decisões arquiteturais**."



Len Bass

Paul Clements

Rick Kazman

PUC Connect

Correção do Exercício Prático:

Proposta de Arquitetura de Software

Especificação do Sistema

A universidade deseja desenvolver um sistema chamado **PUC Connect**, com o objetivo de permitir a comunicação e interação entre alunos e professores.

Alunos:

- Visualizar disciplinas matriculadas
- Enviar mensagens aos professores
- Receber avisos das disciplinas
- Acessar materiais disponibilizados

Professores:

- Enviar avisos
- Responder mensagens
- Publicar materiais

O sistema será acessado via navegador e dispositivos móveis.

Requisitos Não Funcionais (Arquiteturalmente Significativos)

RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

RNF02 — O sistema deve estar disponível via navegador web e dispositivos móveis.

RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.

RNF04 — O sistema deve permitir futura expansão.

RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

RNF06 — O sistema deve possuir disponibilidade mínima de 99%, permitindo no máximo 3,65 dias de indisponibilidade por ano.

1º - Vamos identificar algumas entidades de domínio

Correção do Exercício Prático:

Proposta de Arquitetura de Software

Especificação do Sistema

A universidade deseja desenvolver um sistema chamado **PUC Connect**, com o objetivo de permitir a comunicação e interação entre alunos e professores.

Alunos:

- Visualizar disciplinas matriculadas
- Enviar mensagens aos professores
- Receber avisos das disciplinas
- Acessar materiais disponibilizados

Professores:

- Enviar avisos
- Responder mensagens
- Publicar materiais

Disciplina

Arquivos

Usuário

Comunicação

O sistema será acessado via navegador e dispositivos móveis.

Requisitos Não Funcionais (Arquiteturalmente Significativos)

RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

RNF02 — O sistema deve estar disponível via navegador web e dispositivos móveis.

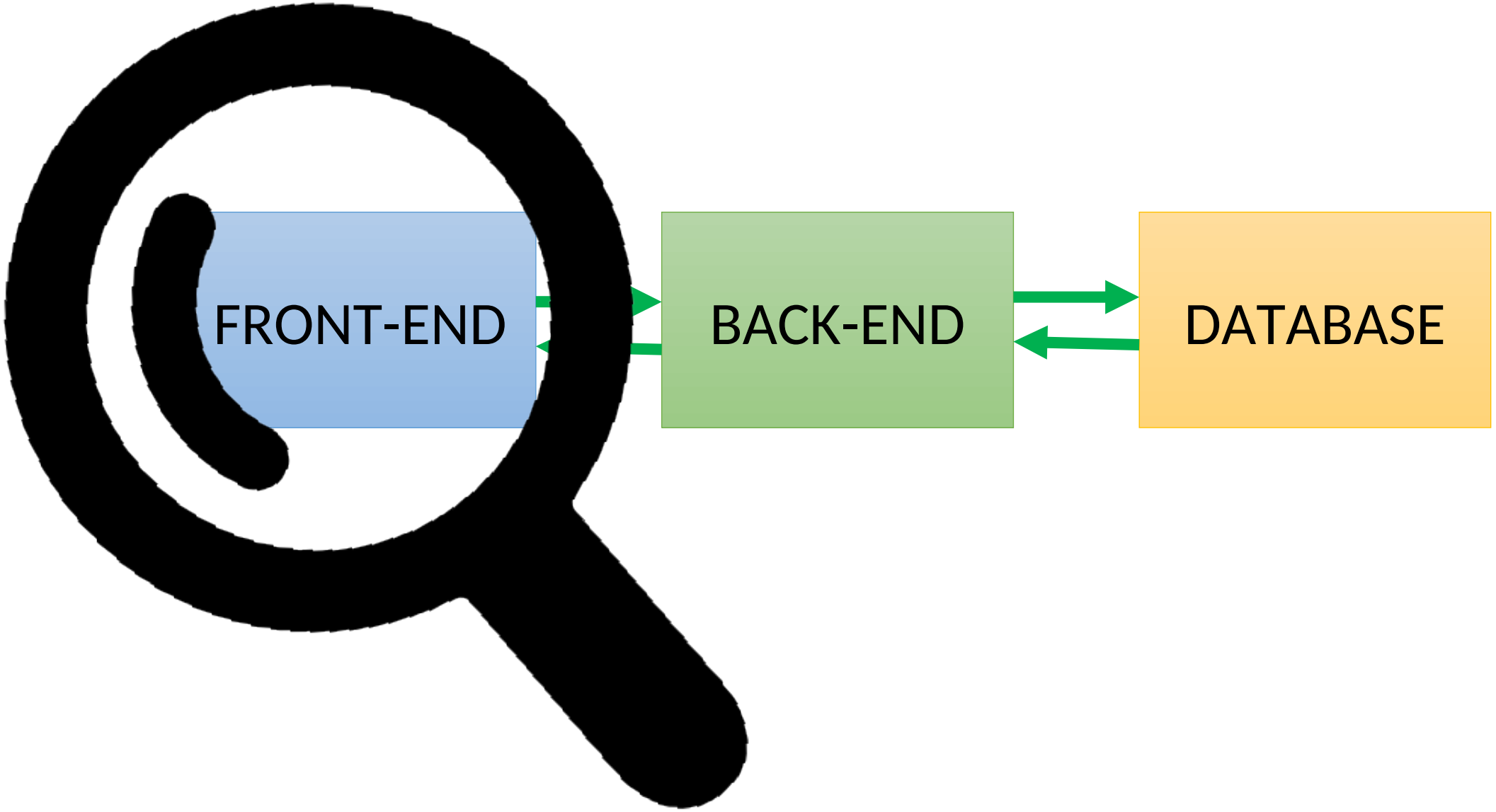
RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.

RNF04 — O sistema deve permitir futura expansão.

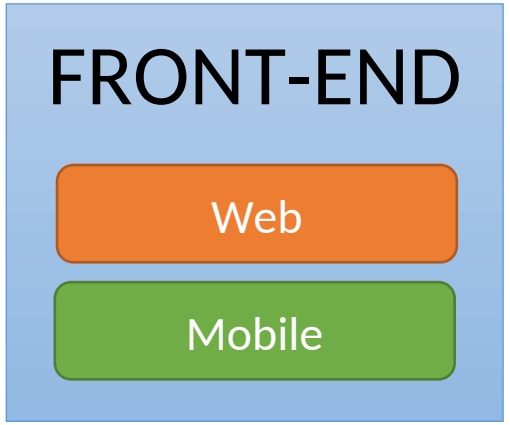
RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

RNF06 — O sistema deve possuir disponibilidade mínima de 99%, permitindo no máximo 3,65 dias de indisponibilidade por ano.

FRONT-END
BACK-END
DATABASE


FRONT-END
BACK-END
DATABASE

RNF02 — O sistema deve estar disponível via navegador **web** e dispositivos **móveis**.

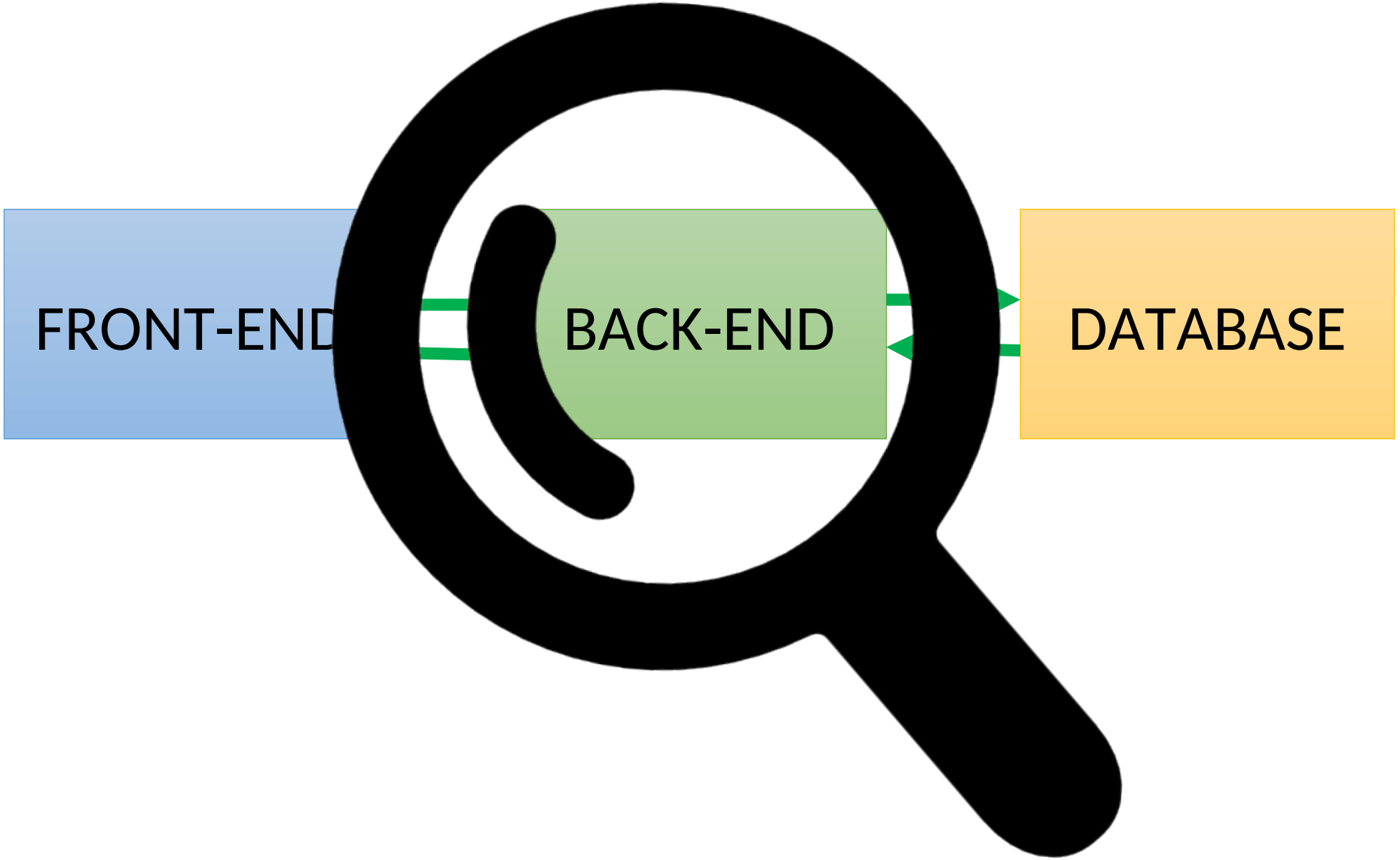


Web

- ❑ ASP.NET
- ❑ ANGULAR
- ❑ PHP
- ❑ REACT
- ❑ **VUE**

Mobile

- ❑ ANDROID (kotlin)
- ❑ IOS (swift)
- ❑ REACT NATIVE
- ❑ WEB APP
- ❑ **FLUTTER**


FRONT-END
BACK-END
DATABASE

RNF04 — O sistema deve permitir futura expansão.

RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

- ❑ NODE
- ❑ JAVA
- ❑ **DOTNET**
- ❑ PHP
- ❑ GO
- ❑ RUBY
- ❑ PYTHON

Futura expansão?

Aumentar o **escopo** do sistema?

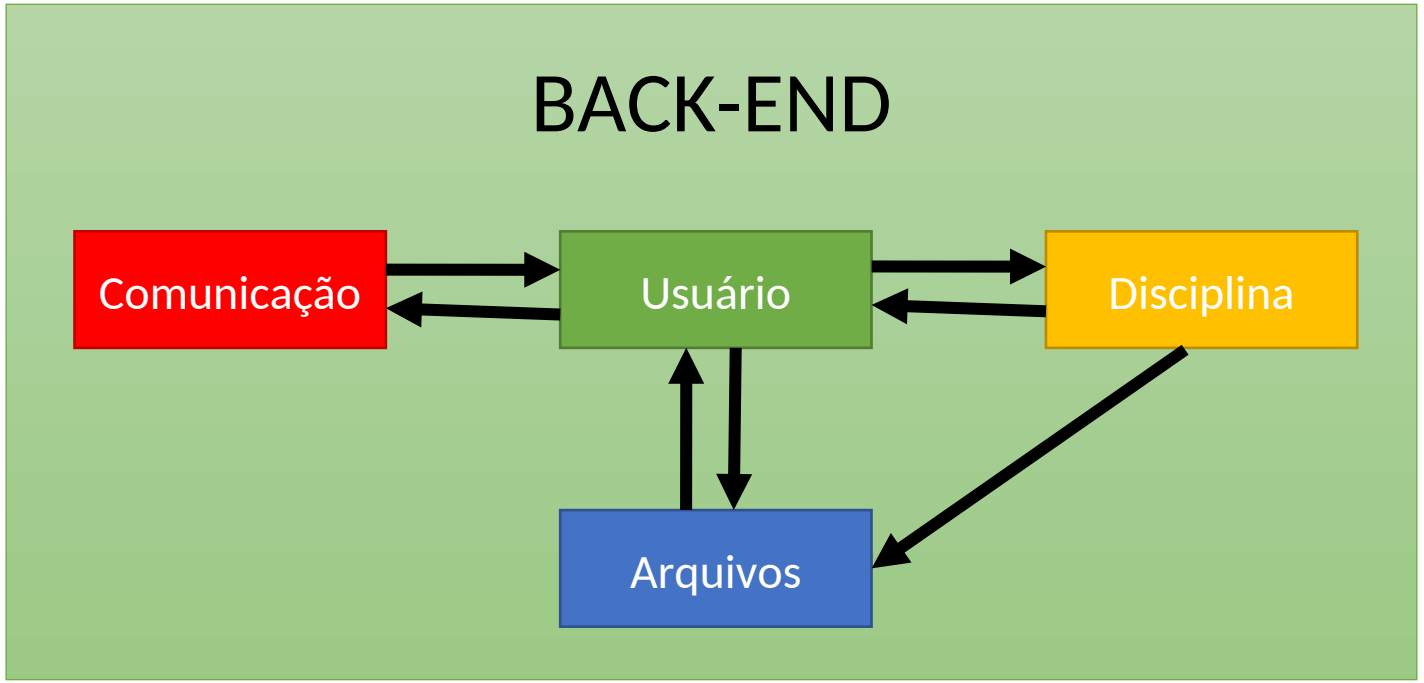
Aumentar a **carga** de usuários?

Vamos considerar ambos!

Tempo de resposta...

- Preciso tomar cuidado com latência
- Preciso evitar leituras desnecessárias no BD
- Preciso construir consultas otimizadas
- Preciso paralelizar os processos sempre que possível
- Preciso balancear a carga para não ter gargalos em picos de uso

Monolito Modular

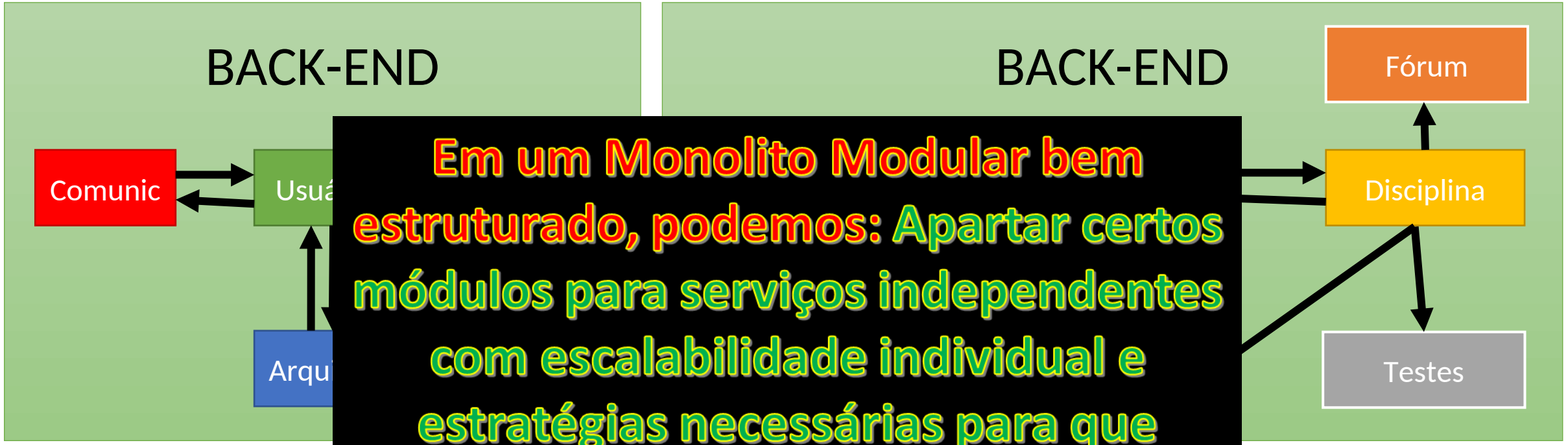


Todos os **módulos na mesma solução** se relacionando por dependência com chamadas de métodos/interface

- Simplicidade e eficiência (sem *overhead* de rede)
- Simplicidade no controle de transações
- Menor complexidade operacional

- Tendência a aumentar acoplamento
  - Limites precisam ser claros
  - Design de Software precisa ser impecável

Monolito Modular



Em um Monolito Modular bem estruturado, podemos: Apartar certos módulos para serviços independentes com escalabilidade individual e estratégias necessárias para que funcione de forma eficiente.

Como podemos ate **tir futura expansão”** em caso de aumento do escopo da aplicação?

E se com isso um ou mais módulos se tornarem um gargalo de performance?

E se a expansão for em termos do número de usuários simultâneos?

Requisitos Não Funcionais (Arquiteturalmente Significativos)

RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

✓RNF02 — O sistema deve estar disponível via navegador web e dispositivos móveis.

RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.

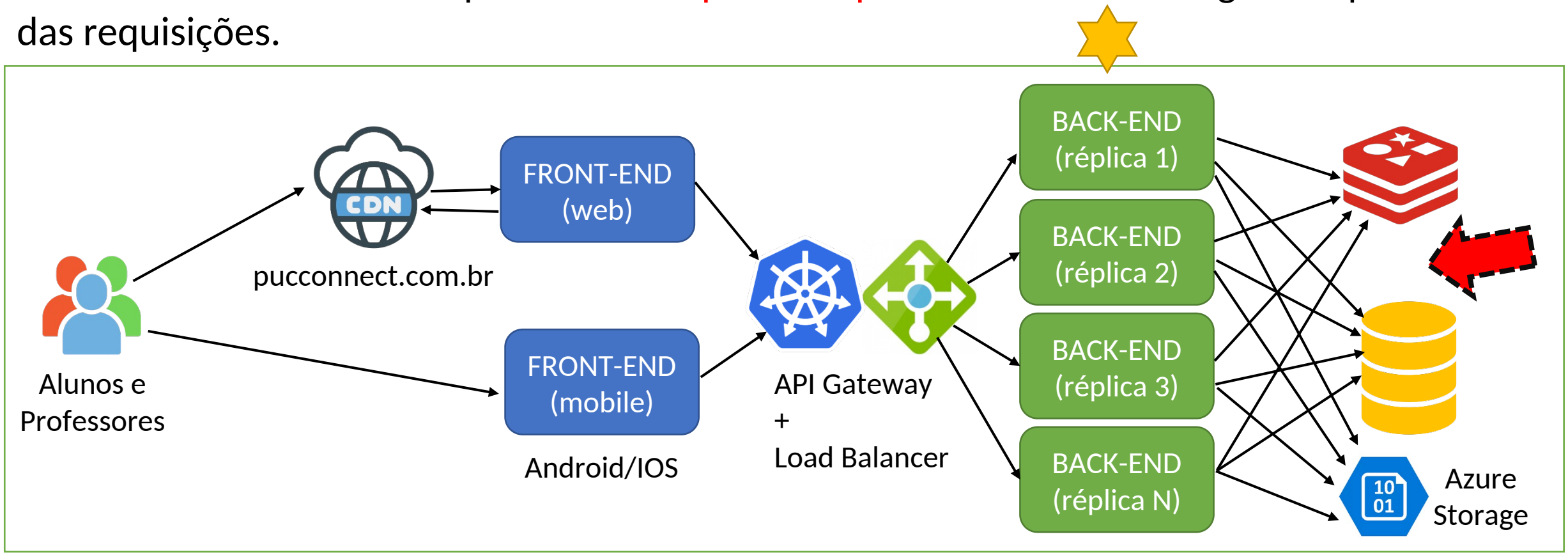
✓RNF04 — O sistema deve permitir futura expansão.

✓RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

RNF06 — O sistema deve possuir disponibilidade mínima de 99%, permitindo no máximo 3,65 dias de indisponibilidade por ano.

RNF05 começamos a preparar o terreno para atender com a estratégia do Monólito Modular que irá reduzir significativamente a latência entre serviços.

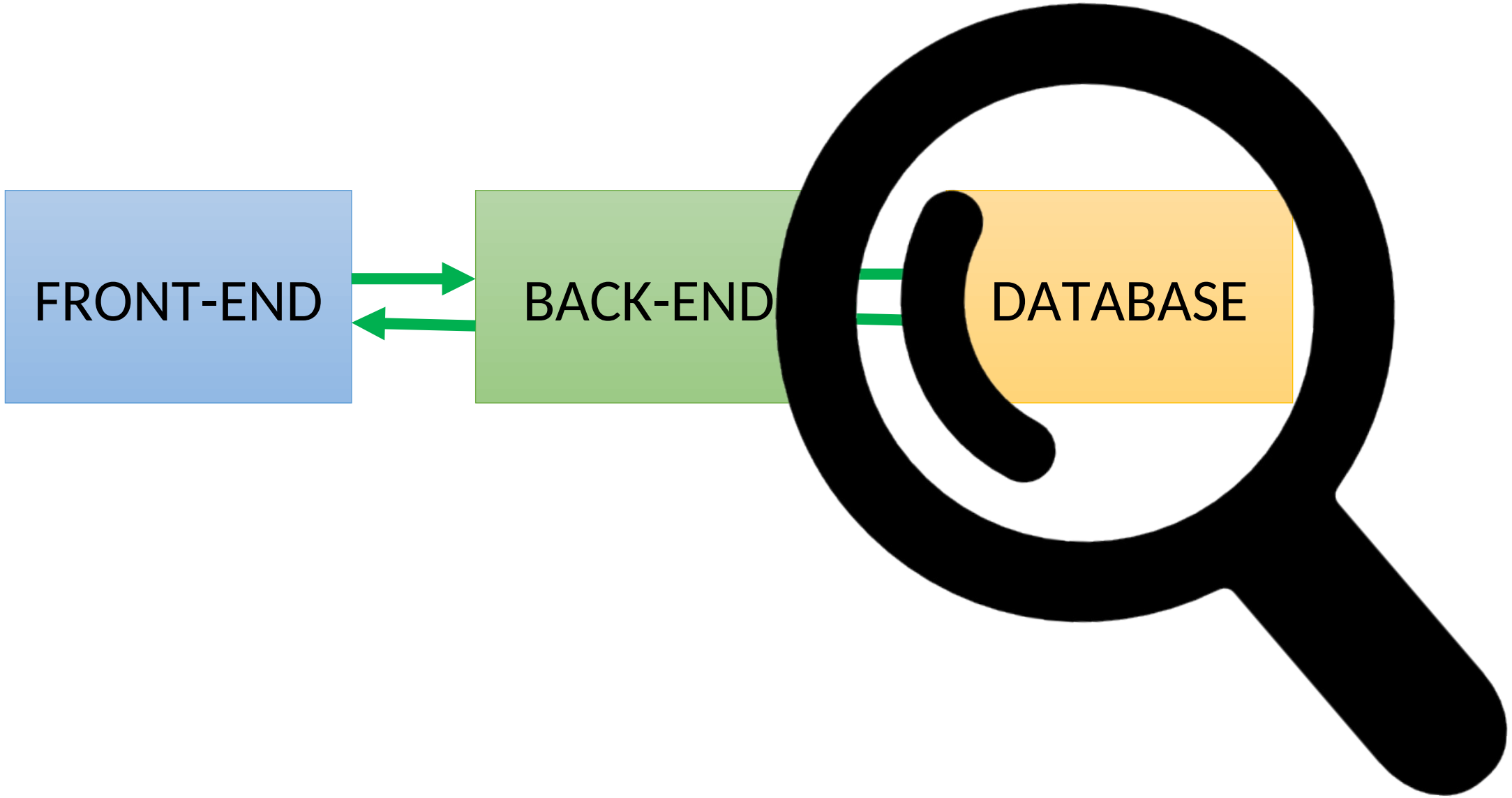
RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.



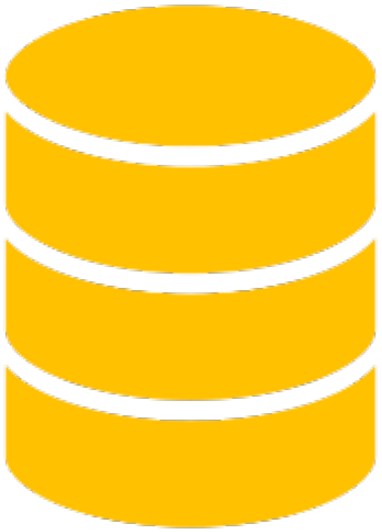
- REDUZIR A LATÊNCIA
  - Monolito Modular vai ajudar com isso
  - Decisões ruins podem prejudicar a latência!
- EVITAR LEITURAS DESNECESSÁRIAS NO BD
  - Cache-aside (ou lazy loading)
- CONSTRUIR QUERIES OTIMIZADAS
  - Banco de dados bem modelado
  - Índices para acelerar consultas
  - Desnormalização estratégica
- AUMENTAR O THROUGHPUT
  - Usar async/await no dotnet
- BALANCEAR CARGA

Já avisei
que isso vai dar merda.

Ainda não iremos falar sobre escalabilidade de Banco de Dados!


FRONT-END
BACK-END
DATABASE

RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.



- ❑ Sql-Server
- ❑ SqLite
- ❑ **PostegreSQL**
- ❑ Oracle
- ❑ MySQL
- ❑ MongoDB
- ❑ DynamoDB
- ❑ Cassandra

Requisitos Não Funcionais (Arquiteturalmente Significativos)

✓ RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

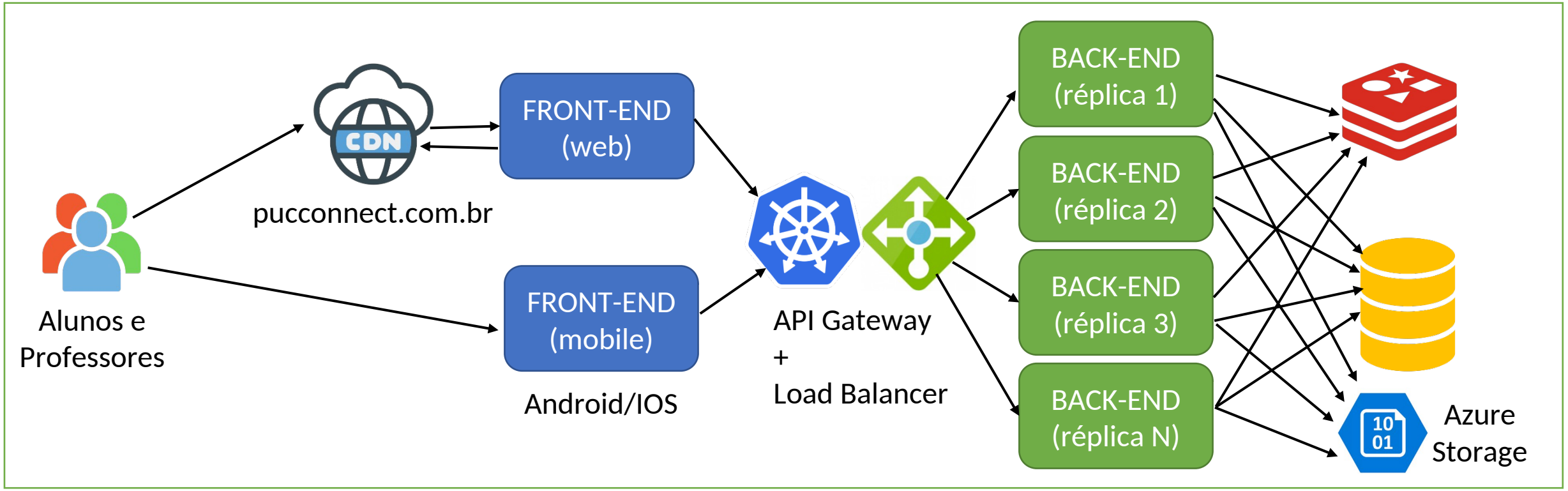
✓ RNF02 — O sistema deve estar disponível via navegador web e dispositivos móveis.

✓ RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.

✓ RNF04 — O sistema deve permitir futura expansão.

✓ RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

✓ RNF06 — O sistema deve possuir disponibilidade mínima de 99%, permitindo no máximo 3,65 dias de indisponibilidade por ano.


CDN
pucconnect.com.br
FRONT-END
(web)
Alunos e
Professores
FRONT-END
(mobile)
Android/IOS
API Gateway
+
Load Balancer
BACK-END
(réplica 1)
BACK-END
(réplica 2)
BACK-END
(réplica 3)
BACK-END
(réplica N)
10
01
Azure
Storage

Requisitos Não Funcionais (Arquiteturalmente Significativos)

✓ RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

✓ RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

Como saber então se minha arquitetura vai suportar esses RNFs?

TESTAR EM PRODUÇÃO????

TESTE DE CARGA!!!

Arquitetura sem teste de carga é opinião!
Arquitetura validada é engenharia!

Conclusão

- Modelagem Arquitetural é a representação estruturada da arquitetura do sistema.
- Arquitetura **não é apenas um desenho**, mas o conjunto de decisões estruturais que atendem aos requisitos arquiteturalmente significativos.
- A qualidade de um modelo arquitetural depende da clareza dos elementos e das relações apresentadas.
- Requisitos não funcionais, como **desempenho, disponibilidade, escalabilidade e segurança**, influenciam diretamente a estrutura arquitetural.
- Uma boa modelagem **deve comunicar decisões**, não apenas componentes.
- Ausência de justificativa arquitetural indica fragilidade na proposta.
- A modelagem permite analisar riscos antes da implementação.
- Arquitetura bem modelada facilita evolução, manutenção e expansão do sistema.
- O objetivo não é desenhar “o mais complexo”, e sim o mais coerente com os requisitos.